Smart Farming Roadmap v2

Smart Farming (SIH 25099) — Project Overview & Implementation Roadmap (v2)

Updated to incorporate: shared context object, crop-specific Decision Engine, workflow status tracking, external configuration, and prediction logging.

1. What the Project Actually Is

Strip away the buzzwords and this is a **4-stage AI pipeline wrapped in a web service**:

1. A farmer uploads a leaf photo (mobile/web).
2. **OpenCV** cleans and validates the image (is it usable? crop out the leaf).
3. A stack of **deep learning models** answer: *what crop, what disease, how bad, any pests?*
4. An **LLM** turns those raw predictions into a farmer-readable recommendation (fertilizer, pesticide, irrigation advice).

Everything else — FastAPI, PostgreSQL, MinIO, React, Docker — exists to move data reliably between these four stages and to store the results. The project is really two disciplines glued together:

- **Data Engineering** = getting a clean, validated, well-structured image (and its metadata) from "farmer's phone" to "model input tensor," and getting predictions from "model output" to "stored record."
- **Data Science** = building the models that sit inside that pipeline (crop ID → disease → severity → pests) and making sure they're accurate and versioned.

2. What Changed From v1 (and Why)

Change	Old Approach	New Approach	Why It's Better
Data passing	Each stage returns a modified copy of the `request` and passes it forward in a straight chain	A single **shared context object** that every stage reads from and writes to	Keeps modules decoupled — any stage can be swapped, skipped, or re-ordered without rewriting the others; much easier to debug because the full state is always inspectable
Disease classification	One disease model for all crops	A **Decision Engine** after crop ID routes to a **crop-specific disease model** (Tomato Disease Model, Potato Disease Model, etc.)	Diseases are crop-specific — a single shared model has to learn many unrelated visual patterns at once, which hurts accuracy. Per-crop models are simpler to train and easier to extend to new crops later
	Implicit — you only	Explicit **status dict** inside the	Makes debugging and monitoring straightforward — you

Progress tracking	know a stage ran if its output field is present	context (`"disease_classification": "pending"` → `"completed"`)	can tell exactly where a request is stuck, and this doubles as data for a future MLOps dashboard
Configuration	Model paths, thresholds, folders hardcoded in scripts	External **YAML config file**	No code changes needed to swap a model version, change a confidence threshold, or move storage locations — this alone is worth showing judges as a maturity signal
Historical data	Predictions stored, but no structured pipeline for it	Every request logs the **full chain**: raw image → processed image → crop → disease → severity → recommendation → farmer feedback	This log *is* your future retraining dataset — it's what makes the MLOps retraining loop possible later

3. Where OpenCV vs. EfficientNet Fit

Role	Tool	Job
Image cleanup	OpenCV	Blur/brightness check, background removal, leaf cropping, CLAHE enhancement, denoising
Classification	EfficientNet (deep learning)	"What crop / what disease is this?"
Explainability	Grad-CAM + OpenCV	Heatmap overlay showing *why* the model predicted that

OpenCV never replaces the model — it's the preprocessing layer that makes the model's job easier and catches garbage input (blurry, cluttered, badly lit photos) before it wastes a model call.

4. Build Order (Do It in This Sequence)

Build bottom-up, not frontend-first:

```
1. OpenCV Preprocessing Service       (standalone, testable on its own)
2. Crop Identification Service        (EfficientNet-B0)
3. Decision Engine                    (routes crop → correct disease model)
4. Disease Classification Services    (one per crop: EfficientNet-B2 / ConvNeXt)
5. Severity Estimation Service        (OpenCV segmentation + area calc)
6. Pest Detection Service             (YOLOv8, optional/stretch)
7. Recommendation Service             (LLM + weather API)
8. Context Object + Pipeline Orchestrator  (glues 1–7 together)
9. Configuration Layer                (YAML config for paths/thresholds)
10. FastAPI Endpoints                 (exposes orchestrator as /predict)
11. Logging Layer                     (full-chain prediction logs)
12. React Dashboard                   (calls /predict, displays results)
```

By step 8, your entire "AI brain" works from a Python script alone — you can test it without any web server.

5. The Shared Context Object

Every stage reads from and writes to the **same context object**, rather than passing a transformed copy down

a chain. This is the single most important architectural decision in the project.

```
        context = {
            "request_id": "REQ001",
            "user": {"user_id": "U001", "location": "Rajkot", "language": "Gujarati"},
            "image": {
                "raw_path": "uploads/img1.jpg",
                "processed_path": None,
                "quality_score": None,
                "blur": None,
                "leaf_detected": None
            },
            "crop": {"label": None, "confidence": None},
            "disease": {"label": None, "confidence": None, "model_used": None},
            "severity": {"percent": None, "affected_area": None},
            "weather": {},
            "recommendation": {},
            "status": {
                "preprocessing": "pending",
                "crop_identification": "pending",
                "decision_routing": "pending",
                "disease_classification": "pending",
                "severity": "pending",
                "recommendation": "pending"
            }
        }
```

Each service function takes the context in, mutates its own section (including its own `status` entry), and returns it:

```
        def preprocess(context):
            context["image"]["processed_path"] = ...
            context["image"]["quality_score"] = ...
            context["status"]["preprocessing"] = "completed"
            return context
```

6. Detailed Flow, Stage by Stage

Stage A — Data Engineering: Ingestion & Validation

```
Farmer uploads image
        │
        ▼
FastAPI receives file → saves to Object Storage (MinIO/S3)
        │
        ▼
Metadata extraction (GPS, timestamp, device info, user_id)
        │
        ▼
Validation: file type check, duplicate check, size check
        │
        ▼
context["image"]["raw_path"] set, context["status"]["preprocessing"] = "pending"
```

Tech: FastAPI, MinIO or AWS S3, Pillow/OpenCV for basic file checks.

Stage B — Data Engineering: OpenCV Pipeline

```
Raw Image
   │
   ├── Blur Detection        → cv2.Laplacian(gray, CV_64F).var()
   ├── Brightness Check       → mean pixel intensity in HSV
   ├── Leaf Detection          → HSV thresholding + contours
   ├── Background Removal      → largest contour mask
   ├── CLAHE Enhancement       → cv2.createCLAHE()
   └── Resize to 224×224
   │
   ▼
context["image"] updated with processed_path, quality_score, blur, leaf_detected
context["status"]["preprocessing"] = "completed"
```

If quality checks fail, short-circuit and ask the farmer to re-upload **before** any model runs.

Tech: OpenCV (`cv2`), NumPy.

Stage C — Data Science: Model Training (done offline, before deployment)

```
Raw Dataset (e.g. PlantVillage or your own collected images)
        │
        ▼
EDA — class balance, image sizes, corrupted files
        │
        ▼
Data Cleaning — remove corrupted/duplicate images
        │
        ▼
Train / Val / Test Split (e.g. 70/15/15, stratified by class)
        │
        ▼
Augmentation — Albumentations (rotation, flip, color jitter, cutout)
        │
        ▼
Transfer Learning — EfficientNet-B0 (crop ID) / per-crop EfficientNet-B2 (disease), pretrained on ImageNet
        │
        ▼
Fine-Tuning — unfreeze later layers, lower LR
        │
        ▼
Evaluation — accuracy, F1, confusion matrix, Grad-CAM sanity checks
        │
        ▼
Best Model → saved as .pth, versioned (e.g. crop_identifier_v2.pth, tomato_disease_v1.pth)
        │
        ▼
Model Registry — path referenced from the YAML config, not hardcoded
```

Maps to your four notebooks: - `01_data_preparation.ipynb` → EDA, cleaning, split, augmentation setup - `02_train_crop_identifier.ipynb` → transfer learning + fine-tuning for crop ID - `03_evaluation.ipynb` → metrics, confusion matrix, Grad-CAM - `04_inference.ipynb` → becomes `services/crop_identifier/predictor.py`'s `predict_crop(context)` function

You'll now also need a **per-crop notebook variant** (or a parameterized version of `02`/`03`) for each disease model — e.g. training the Tomato Disease Model separately from the Potato Disease Model.

Tech: PyTorch, `timm`, Albumentations, scikit-learn, matplotlib, `pytorch-grad-cam`.

Stage D — Crop Identification

```
context["image"]["processed_path"]
      │
      ▼
Crop Identifier (EfficientNet-B0)
      │
      ▼
context["crop"] = {"label": "Tomato", "confidence": 0.99}
context["status"]["crop_identification"] = "completed"
```

Stage E — Decision Engine (new)

```
context["crop"]["label"]
      │
      ▼
Decision Engine
      │
   ┌──┴──┬──────┐
   ▼     ▼      ▼
Tomato  Potato   Cotton
Disease Disease  Disease
Model   Model    Model
      │
      ▼
context["disease"]["model_used"] set
context["status"]["decision_routing"] = "completed"
```

The Decision Engine is simple business logic (a lookup: `crop_label → model_path`), not another AI model. It reads the config file to know which model file to load for the identified crop, and also enforces the confidence threshold (e.g. don't route to disease classification if crop confidence < 0.75 — ask for a clearer photo instead).

Stage F — Disease Classification (crop-specific)

```
context["image"]["processed_path"] + context["disease"]["model_used"]
      │
      ▼
Disease Classifier (EfficientNet-B2 / ConvNeXt, specific to identified crop)
      │
      ▼
context["disease"] = {"label": "Early Blight", "confidence": 0.97, "model_used": "tomato_disease_v1"}
context["status"]["disease_classification"] = "completed"
```

Stage G — Severity Estimation

```
context["image"] + context["disease"]
      │
      ▼
Severity Estimator (OpenCV segmentation: infected area / total leaf area)
      │
      ▼
context["severity"] = {"percent": 32, "affected_area": 18.4}
context["status"]["severity"] = "completed"
```

Stage H — Pest Detection (optional)

```
context["image"]["processed_path"]
      │
      ▼
YOLOv8 → bounding boxes if pests present
      │
      ▼
context["pests"] = [...]
```

Stage I — Recommendation Layer

```
context["crop"] + context["disease"] + context["severity"] + weather + location + crop history
        │
        ▼
Prompt construction
        │
        ▼
LLM API call (Gemini / Llama 3 / Groq)
        │
        ▼
context["recommendation"] = {"fertilizer": ..., "pesticide": ..., "irrigation": ..., "prevention_tips": ...}
context["status"]["recommendation"] = "completed"
```

Tech: Any hosted LLM API + a weather API (OpenWeatherMap is a common free option).

Stage J — Orchestration & API

```python
# pipeline.py — no AI logic here, only sequencing
def run_pipeline(context):
    context = preprocess(context)              # OpenCV
    context = identify_crop(context)            # EfficientNet
    context = route_to_disease_model(context)   # Decision Engine
    context = classify_disease(context)          # crop-specific EfficientNet
    context = estimate_severity(context)         # OpenCV + AI
    context = generate_recommendation(context)   # LLM
    log_prediction(context)                       # full-chain log
    return context
```

```python
# main.py — FastAPI is a thin wrapper
@app.post("/predict")
def predict(file: UploadFile):
    context = create_context(file)
    result = run_pipeline(context)
    save_to_db(result)
    return result
```

Stage K — Configuration (new)

```yaml
# config.yaml
models:
  crop_identifier: models/crop_identifier_v2.pth
```

```yaml
  disease_models:
    tomato: models/tomato_disease_v1.pth
    potato: models/potato_disease_v1.pth
    cotton: models/cotton_disease_v1.pth

thresholds:
  crop_confidence: 0.75
  disease_confidence: 0.70

storage:
  upload_dir: uploads/
  processed_dir: processed/
```

Every service reads its model path and thresholds from this file instead of hardcoding them — swapping a model version becomes a one-line config change, not a code change.

Stage L — Prediction Logging (new)

```
Request
   │
   ▼
Raw Image
   │
   ▼
Processed Image
   │
   ▼
Crop Prediction
   │
   ▼
Disease Prediction
   │
   ▼
Severity
   │
   ▼
Recommendation
   │
   ▼
User Feedback
```

Each full chain is written to the `predictions` + `feedback` tables — this log becomes the training data for future model retraining (the MLOps loop below).

Stage M — Persistence

```
users            (user details, farm details, crop history)
images           (original + processed paths, metadata)
predictions      (crop, disease, severity, model_used, timestamps)
recommendations  (fertilizer, pesticide, irrigation, prevention tips)
feedback         (farmer feedback on prediction accuracy)
```

Tech: PostgreSQL for structured data, MinIO/S3 for image blobs.

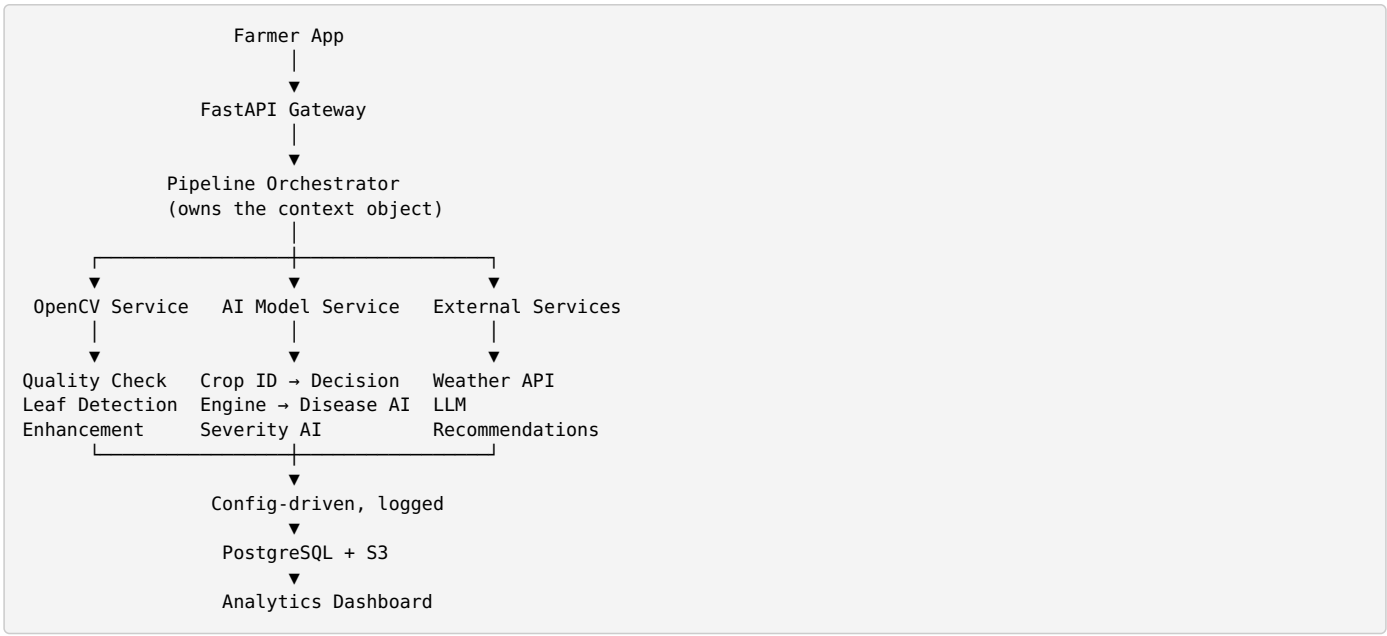
Stage N — Frontend

```
React (Vite) → single API call to /predict → renders crop, disease, severity, recommendation
```

Stage O — MLOps Loop

```
New predictions + farmer feedback (from the logging layer)
        │
        ▼
Performance monitoring (accuracy over time, confidence drift)
        │
        ▼
Drift detection (are new images statistically different from training data?)
        │
        ▼
Retraining trigger → new dataset → retrain → new model version → update config.yaml → redeploy
```

7. Recommended Architecture Diagram



8. Recommended Folder Structure

```
Smart-Farming/
├── backend/
│   ├── main.py
│   ├── pipeline.py              ⭐ orchestrator — no AI logic, owns the context object
│   ├── config.yaml               ⭐ model paths, thresholds, storage config
│   ├── services/
│   │   ├── preprocessing/       (OpenCV)
│   │   ├── crop_identifier/     (predictor.py)
│   │   ├── decision_engine/     (routes crop → disease model)   ⭐ new
│   │   ├── disease_classifier/  (per-crop predictor.py files)
│   │   ├── severity/
│   │   ├── recommendation/      (LLM calls)
│   │   └── weather/
│   ├── models/                  (.pth weights, versioned, referenced from config.yaml)
│   ├── database/
│   │   └── logging.py            ⭐ full-chain prediction logging
│   └── utils/
├── frontend/                    (React/Vite)
├── datasets/
└── notebooks/                   (your 4 notebooks, plus per-crop disease training variants)
```

9. Full Tech Stack Reference

Layer	Tools
Data Engineering	Python, FastAPI, OpenCV, PostgreSQL, MinIO/S3, Pandas, NumPy, PyYAML
Data Science	PyTorch, timm, Albumentations, scikit-learn, matplotlib, Grad-CAM
AI Models	EfficientNet-B0 (crop ID), EfficientNet-B2/ConvNeXt (per-crop disease models), YOLOv8 (pests), optionally Segment Anything (leaf segmentation), Whisper (voice input), Llama 3/Gemini (recommendations)
Deployment	FastAPI, Docker, Nginx, React (Vite), Flutter (optional mobile)

10. Where to Make Changes in Your Existing Notebooks

1. **`01_data_preparation.ipynb`** — insert the OpenCV quality-check step before your train/val/test split;

also separate your disease dataset by crop so you can train per-crop models in step 3.

2. **`02_train_crop_identifier.ipynb`** — confirm `timm.create_model('efficientnet_b0', pretrained=True)` with a classifier head sized to your crop classes; keep this notebook crop-agnostic since it only identifies crop type.
3. **New: `02b_train_disease_<crop>.ipynb`** — one notebook (or a parameterized script) per crop's disease model, following the same transfer-learning pattern as `02`.
4. **`03_evaluation.ipynb`** — add Grad-CAM visualization; extend to evaluate each per-crop disease model separately.
5. **`04_inference.ipynb`** — split into `services/crop_identifier/predictor.py` (`predict_crop(context)`) and `services/decision_engine/router.py` (reads `config.yaml`, picks the right disease model path), followed by `services/disease_classifier/predictor.py` (`predict_disease(context)`).

Everything else (severity, recommendation, orchestrator with context object, config layer, logging, API) is new work layered on top of what these notebooks already give you.